Agentisches Programmieren

Joern Ploennigs · AI4SC



> There's a new kind of coding I call ‚vibe coding', where you fully give in to the vibes, embrace exponentials, and forget that the code even exists.
>
> — Andrej Karpathy, Beitrag auf X, 2025 [1]

Karpathy beschreibt mit *vibe coding* eine Arbeitsweise, bei der wir das Programmieren vollständig einer KI überlassen und den entstandenen Code gar nicht mehr ansehen. Für Ingenieurinnen und Ingenieure ist das keine Option, denn sie verantworten die Ergebnisse ihrer Berechnungen unabhängig davon, wer den Code geschrieben hat. In diesem Kapitel klären wir deshalb, was ein KI-Assistent beim Programmieren tatsächlich tut und wo seine Grenzen liegen.

Zunächst betrachten wir, wie ein großes Sprachmodell Text und Code erzeugt und warum es dabei plausible, aber nicht zwingend korrekte Ergebnisse liefert. Darauf aufbauend lernen wir Agenten und den agentischen Zyklus kennen, der die Arbeitsmethode dieses Kurses bildet, und üben zwei seiner Schritte: einen Prompt als Anforderung zu formulieren und generierten Code gezielt zu lesen. Abschließend fragen wir, wie gut Menschen KI-Ergebnisse überhaupt prüfen können. Als roter Faden dient uns ein einfaches Beispiel aus der Tragwerksplanung, die Biegespannung eines Rechteckträgers.

Fehlerfrei ausgeführt heißt nicht richtig gerechnet

Wir beginnen mit einem Prompt, wie ihn viele von uns in einen Chat-Assistenten tippen würden: „Berechne die Biegespannung eines Rechteckträgers." Der Assistent liefert daraufhin den folgenden Code, der ohne jede Fehlermeldung durchläuft:

```
b, h = 200, 400      # Breite, Höhe in mm
M = 150              # Moment in kNm
W = b * h**2 / 6     # Widerstandsmoment in mm³
```

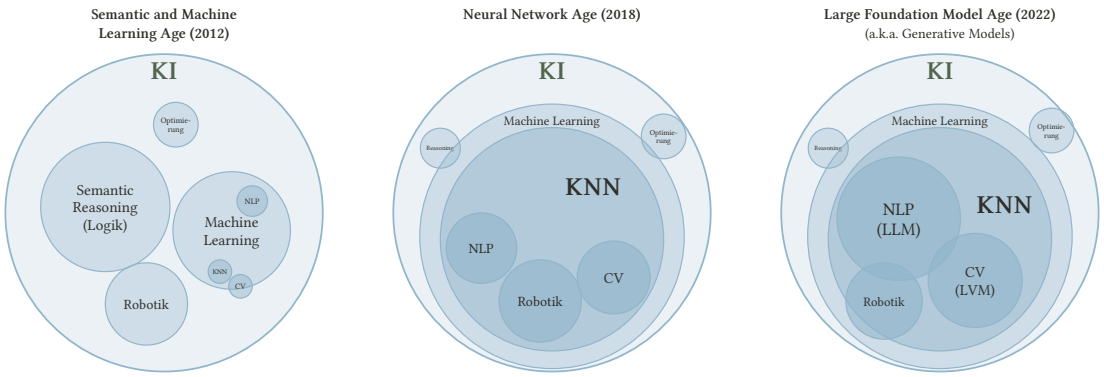
```
sigma = M / W       # Spannung in N/mm²
print(f"Ergebnis: {sigma:.1e}")
```

```
Ergebnis: 2.8e-05
```

Das Ergebnis von 2,8·10⁻⁵ N/mm² sieht zunächst wie eine gewöhnliche Zahl aus. Lesen wir den Code jedoch Zeile für Zeile, fällt auf, dass das Moment in kNm angegeben ist, das Widerstandsmoment hingegen in mm³. Für eine Spannung in N/mm² müsste das Moment zuerst in Nmm umgerechnet, also mit 10^6 multipliziert werden. Mit $W = bh^2/6 = 5\,333\,333\ \text{mm}^3$ und $M = 150 \cdot 10^6$ Nmm ergibt sich richtig $\sigma = M/W$ = 28,125 N/mm². Das Programm liegt also um den Faktor 10^6 daneben, ohne dass Python etwas meldet, denn der Interpreter kennt keine Einheiten und hält jede Zahl für gültig.

Wer die Grundlagen der Statik beherrscht, erkennt den Fehler beim Lesen sofort, beim bloßen Ausführen dagegen nicht. Damit stellt sich die Frage, die uns durch dieses Kapitel begleitet: Warum macht ein so leistungsfähiges System einen so einfachen Fehler, und wie können wir ihn systematisch vermeiden? Um sie zu beantworten, müssen wir zunächst verstehen, wie ein Sprachmodell zu seiner Antwort kommt.

Gebiete der Künstlichen Intelligenz

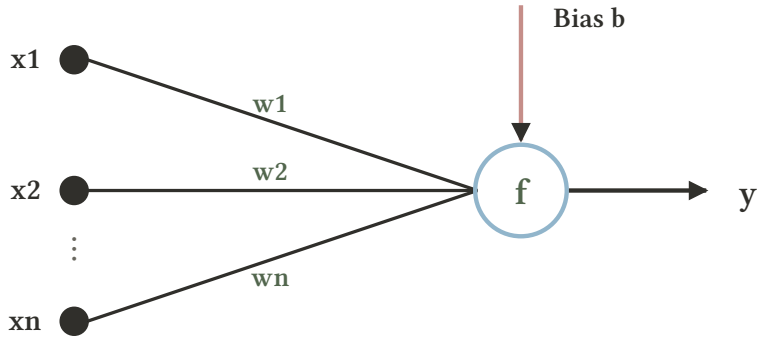


Sprachmodelle sind ein Teilgebiet der Künstlichen Intelligenz (KI), deren Schwerpunkte sich in den letzten Jahren stark verschoben haben. Die Abbildung zeigt drei Phasen dieser Entwicklung. Im Semantic and Machine Learning Age um 2012 standen logikbasiertes Schließen (Semantic Reasoning), Machine Learning, Optimierung und Robotik als weitgehend eigenständige Gebiete nebeneinander. Im Neural Network Age um 2018 wurde Machine Learning zum dominierenden Gebiet, und künstliche neuronale Netzwerke (KNN) durchdrangen die Sprachverarbeitung (Natural Language Processing, NLP), die Bildverarbeitung (Computer Vision, CV) und die Robotik.

Seit etwa 2022, im Large Foundation Model Age, bestimmen große generative Modelle das Feld: Large Language Models (LLM) für Sprache und Large Vision Models (LVM) für Bilder. Alle KI-Werkzeuge, mit denen wir in diesem Kurs programmieren, gehören zu dieser letzten Phase. Ihre gemeinsame Grundlage sind neuronale Netzwerke, die wir uns deshalb als Erstes ansehen.

KNN – Künstliche Neuronale Netzwerke (1943)

Ein künstliches neuronales Netzwerk (KNN) ist ursprünglich ein mathematisches Modell der Nervenzellen im menschlichen Gehirn, das bereits 1943 formuliert wurde. Sein Grundbaustein, das künstliche **Neuron**, ist bemerkenswert einfach aufgebaut. Es multipliziert jeden Eingang x_i mit einem Gewicht w_i, addiert die Produkte sowie einen Bias b und gibt die Summe an eine Aktivierungsfunktion $f(\cdot)$ weiter, die entscheidet, wie stark das Neuron aktiviert wird:

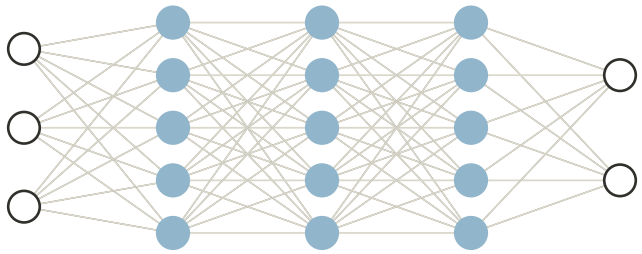


$$y = f\left(b + \sum_{i=1}^{n} w_i x_i\right)$$

Die Gewichte drücken aus, wie wichtig ein Eingang für das Ergebnis ist. Beim Training werden Gewichte und Bias so lange angepasst, bis das Netzwerk für bekannte Beispiele die gewünschten Ausgänge liefert; das Netzwerk lernt also, welche Eingänge zählen. Die Stärke des Ansatzes liegt nicht im einzelnen Neuron, sondern darin, dass sehr viele dieser gleichartigen Bausteine schnell und hochparallel berechnet werden können. Für unsere Ausgangsfrage ist schon hier eine Eigenschaft wichtig: Ein Neuron gewichtet und addiert nur Zahlen, eine Vorstellung von Einheiten, Kräften oder Momenten enthält es nicht.

DNN – Tiefe Neuronale Netzwerke (seit 2011 skalierbar)

Ein einzelnes Neuron kann nur einfache Zusammenhänge abbilden. Tiefe neuronale Netzwerke (Deep Neural Networks, DNN) schalten deshalb viele Schichten von Neuronen hintereinander, je nach Architektur zwischen 3 und 150. Da jedes Neuron einer Schicht mit den Neuronen der nächsten Schicht verbunden ist, wächst die Zahl der Parameter mit jeder Schicht multiplikativ. Praktisch trainieren ließen sich solche Netze erst ab etwa 2011, als zwei Entwicklungen zusammenkamen: Grafikkarten mit tausenden Recheneinheiten, die die vielen gleichartigen Rechnungen parallel ausführen, und bessere Algorithmen, die eine sehr große Zahl von Parametern zuverlässig bestimmen.

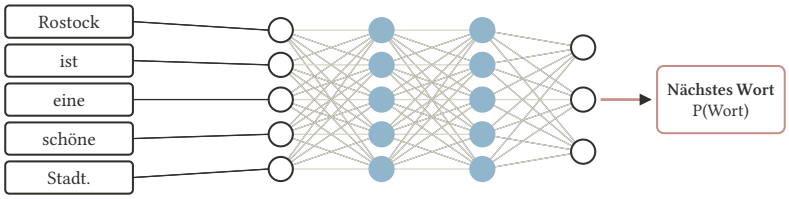


Eingabeschicht Verdeckte Schichten Ausgabeschicht

Der Gewinn der Tiefe liegt darin, dass jede Schicht die Ausgaben der vorherigen zusammenführt. Dadurch kann das Netzwerk Wissen abstrahieren, weil die Eingänge Schicht für Schicht zu allgemeineren Merkmalen verdichtet werden. Zugleich kann es Wissen verknüpfen, weil jede weitere Schicht die abstrahierten Merkmale kombiniert und das für die Aufgabe Wichtige stärker gewichtet. Bei einem Foto einer Betonoberfläche können frühe Schichten beispielsweise Kanten erkennen und spätere daraus den Verlauf eines Risses.

Große Sprachmodelle (Large Language Models LLM) (seit 2017)

Große Sprachmodelle (Large Language Models, LLM) übertragen dieses Prinzip auf Text und vergrößern die Netzwerke auf mehrere Milliarden Parameter. Sie zerlegen Texte in Wörter oder Wortteile und lernen aus sehr großen Textsammlungen, mit welcher Wahrscheinlichkeit welches Wort auf welche vorangehenden Wörter folgt. Möglich wurde das 2017 durch die Transformer-Architektur [2], deren Attention-Mechanismus für jedes Wort gewichtet, welche anderen Wörter im Text für dieses Wort relevant sind, auch über Satzgrenzen hinweg.



Eingabe: Wörter als Zahlen

Auch hier lernt das Netzwerk zu abstrahieren und zu verknüpfen: Es erkennt, welche Wörter in einem Text wichtig sind, und welche anderen Wörter über Sätze hinweg mit ihnen zusammenhängen. Stellen wir eine Anfrage an einen Chat-Assistenten wie ChatGPT, wird dieses gelernte Wissen umgekehrt genutzt, indem das Modell die Wortfolge erzeugt, die als Fortsetzung unserer Anfrage am wahrscheinlichsten ist. Quellcode ist für das Modell ebenfalls nur Text, und auf genau diese Weise ist auch der Code aus unserem Einstiegsbeispiel entstanden.

Ein Sprachmodell sagt das nächste Wort voraus

Wie entsteht aus diesen Wahrscheinlichkeiten ein ganzer Text? Ein LLM verarbeitet intern keine Buchstaben, sondern Zahlen. Jedes Wort oder Wortteil, ein sogenanntes **Token**, steht in einem

festen Vokabular und hat dort eine Nummer. Die Nummern des bisherigen Textes laufen durch das neuronale Netzwerk, das am Ende für jedes Token des Vokabulars eine Wahrscheinlichkeit ausgibt. Das Modell wählt daraus das nächste Token, hängt es an den Text an und beginnt mit dem verlängerten Text von vorn. Dieses Vorgehen heißt *autoregressive Generierung*.

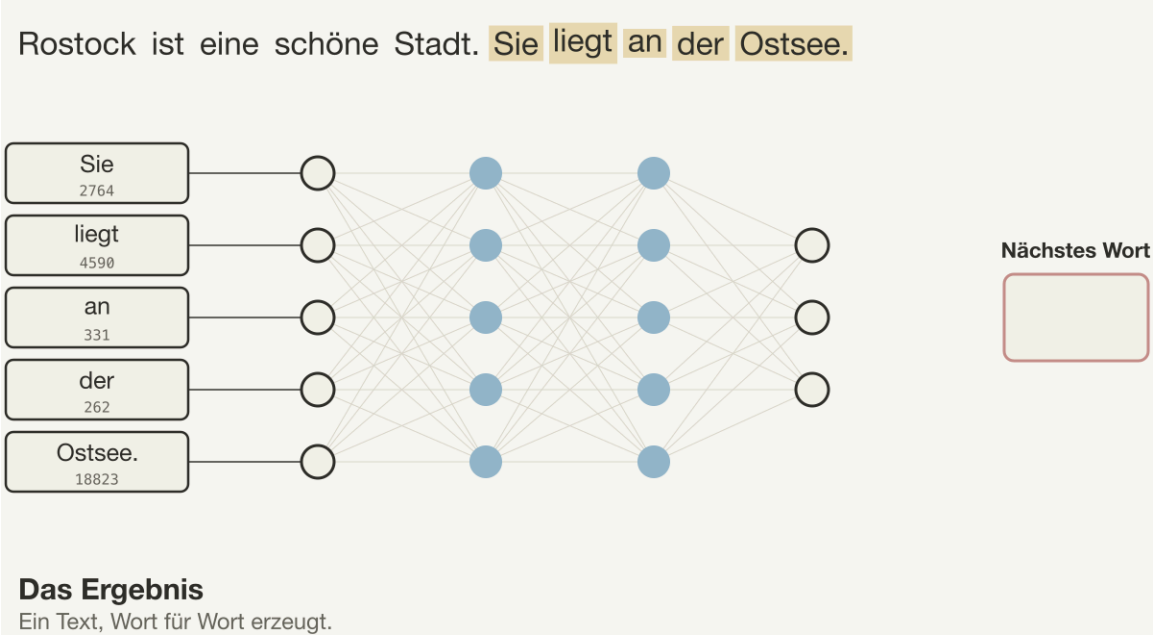


Abbildung 1: Das Netzwerk liest die Nummern der Wörter und gibt das nächste Wort aus, das an den Text angehängt wird

Die Abbildung zeigt den Ablauf an einem Beispiel, wobei die Nummern illustrativ gewählt sind: Aus „Rostock ist eine schöne Stadt." entsteht Wort für Wort die Fortsetzung „Sie liegt an der Ostsee." Entscheidend ist, dass das Modell in jedem Schritt nur das nächste Token vorhersagt. Es plant keine Rechnung und prüft am Ende kein Ergebnis. Ob eine Zeile wie `sigma = M / W` mit passenden Einheiten arbeitet, spielt für die Vorhersage daher keine Rolle, solange der Code wie typischer Code aussieht.

Warum ist das ein Problem?

Damit wird deutlich, warum das Vorgehen eines Sprachmodells für Ingenieuraufgaben problematisch ist. Eine Ingenieurin, die die Biegespannung berechnet, übersetzt die Aufgabe in ein mentales Modell: Sie stellt sich den Träger, die Belastung und die Spannungsverteilung im Querschnitt vor. In diesem Modell versteht sie, warum und wann sich der Balken biegt, kann daraus kausale Schlüsse ziehen und das Ergebnis auf physikalische Plausibilität prüfen. Eine Randspannung von 0,000028 N/mm² unter einem Moment von 150 kNm würde ihr deshalb sofort als unrealistisch auffallen.

Ein LLM übersetzt dieselbe Aufgabe dagegen in ein sprachliches Muster. Es weiß, dass Wörter wie „Balken", „biegen" und „Widerstandsmoment" häufig gemeinsam vorkommen und welche Codezeilen typischerweise darauf folgen, besitzt aber kein Modell der dahinterliegenden Mechanik.

Es sagt nur statistisch wahrscheinliche Wörter voraus und kann nicht prüfen, ob seine Ausgabe sinnvoll ist; im wörtlichen Sinn rechnet es also nicht [3].

Das Training belohnt Plausibilität, nicht Korrektheit

Woher kommt dieses Verhalten? Ein Sprachmodell durchläuft drei Trainingsphasen. Beim *Pre-Training* lernt es an sehr großen Sammlungen aus Text und Code, die jeweils nächste Fortsetzung vorherzusagen, wobei die Verlustfunktion allein misst, wie gut die tatsächliche Fortsetzung getroffen wurde. Beim *Instruction-Tuning* ahmt es Paare aus Aufgaben und Musterantworten nach und lernt so, Anweisungen zu befolgen. In der Feedback-Phase bewerten Menschen alternative Antworten, und das Modell wird auf die bevorzugten Antworten ausgerichtet [4].

In keiner dieser Phasen kommen eine Einheitenprüfung, das Ergebnis einer Programmausführung oder die konkrete Anforderung der späteren Nutzenden vor. Belohnt wird, was plausibel und hilfreich klingt, nicht was nachweislich korrekt ist. Dabei ist das Modell kein Nachschlagewerk, das gespeicherten Code wiedergibt, denn es setzt aus gelernten Mustern auch Code zusammen, der so nie in den Trainingsdaten stand. Darin liegt seine Stärke, weil es für fast jede Aufgabe einen Lösungsvorschlag liefern kann. Darin liegt aber auch die Ursache unseres Einheitenfehlers: `sigma = M / W` ist ein häufiges und plausibles Muster, und nichts im Training hat das Modell gezwungen, die Einheit von `M` zu hinterfragen.

Programmcode ist deterministisch und prüfbar, ein Sprachmodell nicht

Der Unterschied wird deutlicher, wenn wir ein Sprachmodell mit einem gewöhnlichen Programm vergleichen. Ein Programm erhält Daten als Eingabe und verhält sich deterministisch: Dieselbe Eingabe liefert immer dieselbe Ausgabe, und ein Syntax- oder Typfehler führt zu einer Fehlermeldung mit Zeilenangabe. Ob die Formel im Programm fachlich stimmt, garantiert das zwar nicht, wie unser Einstiegsbeispiel zeigt. Das Verhalten ist aber reproduzierbar und lässt sich deshalb systematisch testen. Ähnlich verlässlich arbeiten eine Datenbankabfrage, die genau die gespeicherten Datensätze zurückgibt, oder ein Compiler, der Quellcode nach festen Regeln übersetzt.

Ein Sprachmodell erhält dagegen einen Prompt und gibt keine formale Garantie für seine Ausgabe. Es antwortet immer, auch wenn der Prompt unklar ist oder die richtige Antwort nie gelernt wurde, und dieselbe Eingabe kann bei erneuter Anfrage eine andere Ausgabe liefern. Statt einer Fehlermeldung erhalten wir im Fehlerfall eine plausibel klingende, aber falsche Antwort. Die Verantwortung für die Korrektheit liegt damit vollständig bei den Menschen, die das Modell nutzen, und Verifikation wird vom optionalen Zusatzschritt zur Pflicht.

Kann KI das allein? → Die Kompetenzfalle

Wie gut lösen Sprachmodelle Ingenieuraufgaben, wenn wir sie allein arbeiten lassen? Die Antwort hängt davon ab, welche Fähigkeit gefragt ist. Beim Verstehen sind sie stark im Zusammenfassen und im Allgemeinwissen zu Bauen und Architektur; ihnen fehlen jedoch numerisches und räumliches Verständnis sowie Detailwissen zur Baudomäne und zum konkreten Projekt. Beim Generieren übersetzen sie zuverlässig und erzeugen Bilder, halluzinieren aber bei Unwissenheit und liefern keine wiederholbar zusammenhängenden Ergebnisse. Ihr Wissen lässt sich durch

Retrieval-Augmented Generation (RAG) erweitern oder durch Nachtraining auf Spezialwissen abstimmen, an komplexe Ingenieurprobleme lassen sie sich damit aber dennoch nicht anpassen.

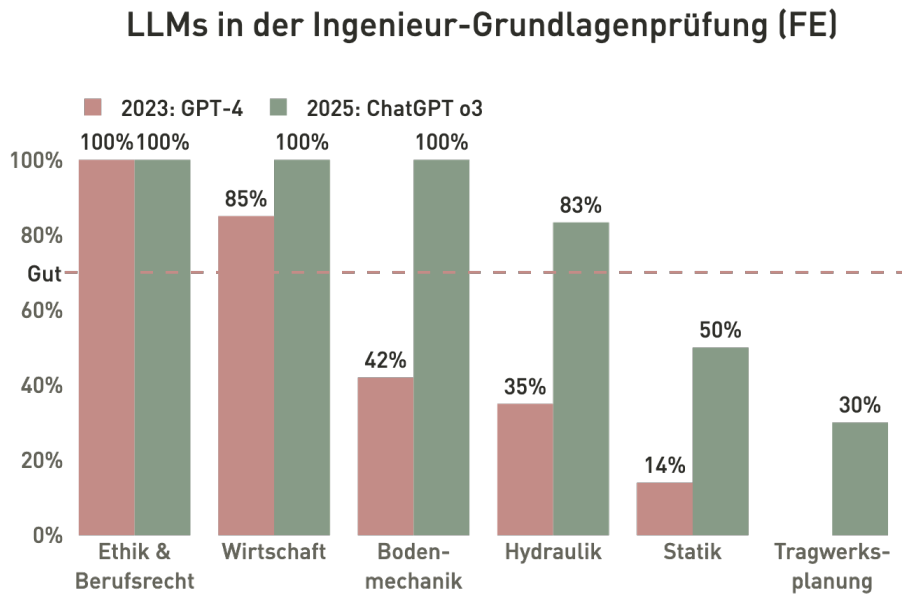


Abbildung 2: LLMs in der Ingenieur-Grundlagenprüfung (FE), Anteil richtig gelöster Aufgaben je Fachgebiet [5], [6]

Das Diagramm zeigt diese Grenzen anhand der US-amerikanischen Ingenieur-Grundlagenprüfung (Fundamentals of Engineering, FE). Auf der x-Achse stehen die Fachgebiete, auf der y-Achse der Anteil richtig gelöster Aufgaben, und die gestrichelte Linie bei 70 Prozent markiert ein gutes Ergebnis. GPT-4 erreichte 2023 in Ethik und Berufsrecht 100 Prozent und in Wirtschaft 85 Prozent, in Bodenmechanik, Hydraulik und Statik dagegen nur 42, 35 und 14 Prozent [5]. Das neuere Modell ChatGPT o3 verbesserte sich 2025 deutlich, blieb aber in der Statik bei 50 und in der Tragwerksplanung bei 30 Prozent [6]. Gerade die Gebiete, die unser Beispiel betrifft, bleiben also die schwächsten. Wir sprechen von einer Kompetenzfalle, weil die sprachliche Gewandtheit des Modells eine fachliche Kompetenz nahelegt, die es in diesen Gebieten nicht besitzt.

Agentisches Programmieren

Code-Vervollständigung wurde in acht Jahren zum Agenten

Nachdem wir verstanden haben, wie ein Sprachmodell Code erzeugt, wenden wir uns den Werkzeugen zu, mit denen wir in diesem Kurs arbeiten. Die Technik hinter ihnen ist erstaunlich jung, wie die folgende Übersicht zeigt.

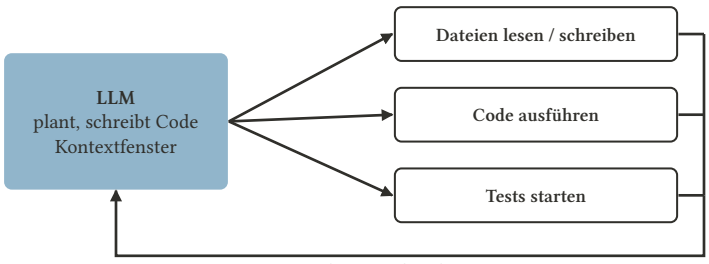
Jahr	Meilenstein	Bedeutung
2017	Transformer [2]	Attention ersetzt rekurrente Netze

Jahr	Meilenstein	Bedeutung
2020	GPT-3 [7]	175 Mrd. Parameter, lernt Aufgaben aus wenigen Beispielen im Prompt
2021	Codex und GitHub Copilot [8]	Sprachmodell auf Quellcode feinjustiert
2022	ChatGPT [4]	Instruction-Tuning und Feedback von Menschen
2025	Coding-Agenten	Sprachmodell mit Zugriff auf Dateien und Terminal

Die ersten Code-Werkzeuge vervollständigten einzelne Zeilen im Editor. Chat-Assistenten konnten ab 2022 ganze Funktionen auf Anfrage schreiben, die wir anschließend selbst in unser Projekt kopierten. Seit 2025 verbreiten sich Coding-Agenten, die Dateien selbst lesen und ändern, Code ausführen und Tests starten. Mit jeder dieser Stufen übernimmt die KI mehr Arbeit, und umso größer wird die Versuchung, ihre Ergebnisse ungeprüft zu übernehmen, also genau das *vibe coding*, das Karpathy beschreibt [1].

Ein Agent ist ein Sprachmodell in einer Schleife mit Werkzeugen

Ein Chat-Assistent beantwortet eine Anfrage und wartet dann auf die nächste. Ein **Agent** erweitert das Sprachmodell dagegen um eine Schleife mit Werkzeugen. Das Modell plant einen Schritt, ruft ein Werkzeug auf, etwa zum Lesen oder Schreiben einer Datei, zum Ausführen von Code oder zum Starten von Tests, und erhält dessen Ergebnis zurück in sein Kontextfenster. Auf dieser Grundlage plant es den nächsten Schritt. So arbeitet ein Agent mehrere Schritte selbstständig ab und kann beispielsweise eine Fehlermeldung lesen und den Code daraufhin korrigieren.

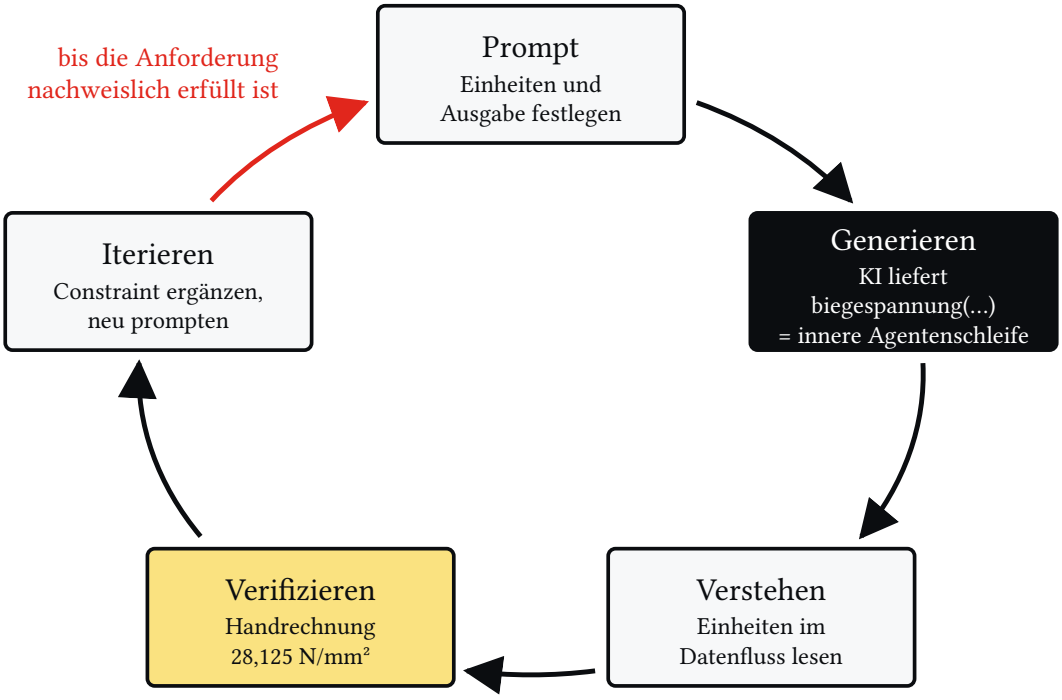


Der Agent schleift selbstständig.
Die Anforderung kennt nur der Mensch.

Die gängigen Werkzeuge unterscheiden sich vor allem im Grad dieser Autonomie, und damit ändert sich, was wir prüfen müssen. GitHub Copilot ergänzt Zeilen während des Tippens; geprüft wird jeder einzelne Vorschlag, bevor wir ihn mit der Tabulatortaste übernehmen, was bequem ist, aber dazu verleitet, Code ungelesen zu akzeptieren. Cursor kombiniert Editor und Chat und

schlägt Änderungen vor, die wir als Diff vor der Übernahme durchsehen. Claude Code, das Werkzeug dieses Kurses, arbeitet im Terminal oder als VS-Code-Erweiterung mit Sicht auf das ganze Projekt; es plant, liest, führt aus und testet selbst, sodass wir am Ende das Ergebnis gegen die Anforderung prüfen. Je höher die Autonomie, desto wichtiger wird diese Prüfung, denn der Agent kennt unsere Anforderung nur aus dem Prompt. Selbst wenn er Tests ausführt, kann er nicht wissen, ob diese Tests die richtige Anforderung abbilden.

Der agentische Zyklus legt eine äußere Schleife um den Agenten



Der Agent arbeitet seine innere Schleife selbstständig ab, doch die Anforderung kennt nur der Mensch. Deshalb legen wir um diese innere Schleife eine äußere, den **agentischen Zyklus**, der die Arbeitsmethode des ganzen Kurses bildet. Er besteht aus fünf Schritten: Prompt, Generieren, Verstehen, Verifizieren und Iterieren. Nur das Generieren übernimmt die KI, die übrigen vier Schritte liegen bei uns.

Am Leitbeispiel lässt sich jeder Schritt konkret benennen. Im Prompt legen wir Einheiten und Ausgabegröße fest. Die KI generiert daraufhin eine Funktion `biegespannung(…)` und durchläuft dabei gegebenenfalls selbst die innere Agentenschleife. Beim Verstehen lesen wir den Code und verfolgen die Einheiten im Datenfluss. Beim Verifizieren vergleichen wir das Ergebnis mit einer unabhängigen Handrechnung, die 28,125 N/mm² ergibt. Weicht es ab, ergänzen wir beim Iterieren den fehlenden Constraint und prompten neu, bis die Anforderung nachweislich erfüllt ist. Jeder dieser Schritte verlangt eine eigene Kompetenz, nämlich Anforderungen zu formulieren, Code zu lesen und in Tests zu denken; genau diese Fähigkeiten baut der Kurs auf.

Der Zyklus ist die Methode des ganzen Kurses

Block	Thema	Trainierter Schritt
1	Computational Foundations	Verstehen
2	Computational Engineering	Prompt
3	Programming Foundations	Verstehen, Iterieren
4	Verification & Quality	Verifizieren
5	Problem Classes	Prompt, Verifizieren
6	Data Management	ganzer Zyklus

Die Tabelle zeigt, wie sich der agentische Zyklus über die sechs Blöcke des Kurses (S01 bis S13) verteilt. Im ersten Block steht das Verstehen im Vordergrund, denn wir lernen, Code zu lesen und Datentypen zu erkennen. Der zweite Block, zu dem dieses Kapitel gehört, trainiert die Formulierung von Anforderungen als Prompt. Im dritten Block lesen und überarbeiten wir Kontrollstrukturen und Funktionen, während der vierte Block die Verifikation durch Unit Tests und Code Review formalisiert. Im fünften Block wenden wir Prompt und Verifikation auf typische Problemklassen an, und der sechste Block verbindet alle Schritte in datenbankgestützten Projekten. Die Zuordnung ist als Orientierung gedacht, denn jede Sitzung kann auch weitere Schritte berühren.

Ein Prompt ist eine Anforderungsspezifikation

Der erste Schritt des Zyklus entscheidet maßgeblich darüber, wie viele Iterationen folgen. Ein Prompt ist im Kern eine Anforderungsspezifikation, wie wir sie aus der Softwareentwicklung kennen, und sollte fünf Elemente enthalten. Die **Rolle** legt fest, aus welcher fachlichen Perspektive die KI antworten soll, hier als Python-Entwickler für Tragwerksplanung. Der **Kontext** beschreibt die Ausgangslage, insbesondere die Eingabegrößen und ihre Einheiten. Die **Aufgabe** benennt präzise, was entstehen soll, möglichst mit Funktionsname und Signatur. Ein **Constraint** schränkt die Lösung ein, etwa auf eine interne Rechnung in N und mm oder auf den Verzicht auf externe Bibliotheken. Das **Ausgabeformat** bestimmt schließlich die Form des Ergebnisses, zum Beispiel einen Docstring mit Einheiten und Typ-Hints.

Für unser Beispiel ergibt sich daraus der folgende Prompt:

> „Du bist Python-Entwickler für Tragwerksplanung.
> Eingaben: b und h in mm, M in kNm.
> Schreibe `biegespannung(b_mm, h_mm, M_knm) -> float`.
> Rechne intern in N und mm, Ergebnis in N/mm².
> Keine externen Bibliotheken.
> Mit Docstring, der die Einheiten nennt, und Typ-Hints.“

Der vage Prompt „Berechne die Biegespannung eines Rechteckträgers“ ließ dagegen alle diese Punkte offen. Die KI musste Einheiten, Funktionsnamen, Rückgabewert und erlaubte

Hilfsmittel erraten, und sie riet plausibel, aber nicht passend. Je präziser der Prompt, desto weniger Iterationen sind nötig; wie wir Anforderungen systematisch erheben, vertiefen wir in `04a_Anforderungen`.

Der fehlende Constraint verursacht den Einheitenfehler

Wie stark der Prompt das Ergebnis bestimmt, zeigt der direkte Vergleich zweier Varianten. Der vage Prompt „Schreibe eine Python-Funktion für die Biegespannung eines Rechteckquerschnitts." nennt keine Einheiten und führt zu einer Funktion wie dieser:

```python
def biegespannung_v1(b, h, M):
    W = b * h**2 / 6
    return M / W

print(f"{biegespannung_v1(200, 400, 150):.1e}")
```

```
2.8e-05
```

Die Funktion läuft und gibt einen Wert aus, der um den Faktor 10^6 zu klein ist, weil sie das Moment in kNm unmittelbar durch das Widerstandsmoment in mm³ teilt. Legt der Prompt dagegen Signatur, Eingabeeinheiten und Ergebniseinheit fest („Schreibe `biegespannung(b_mm, h_mm, M_knm)`. b und h in mm, M in kNm. Rechne intern in N und mm, Ergebnis in N/mm²."), entsteht eine Funktion, deren Parameternamen die Einheiten bereits tragen:

```python
def biegespannung(b_mm, h_mm, M_knm):
    """Biegespannung in N/mm²; M in kNm."""
    W_mm3 = b_mm * h_mm**2 / 6
    return M_knm * 1e6 / W_mm3

print(f"{biegespannung(200, 400, 150):.3f}")
```

```
28.125
```

Hier steht die Umrechnung `M_knm * 1e6` explizit im Code, und der Docstring nennt die Einheit des Ergebnisses, sodass die Funktion die erwarteten 28,125 N/mm² liefert. Beide Funktionen sind syntaktisch einwandfrei und laufen ohne Fehlermeldung; der Unterschied liegt allein in der Spezifikation. Der Einheitenfehler ist daher weniger ein Versagen des Modells als die Folge eines unvollständigen Prompts, und für die vollständige Spezifikation sind wir als Ingenieurinnen und Ingenieure verantwortlich.

Lesen Sie zuerst, was die Funktion verspricht, dann wie sie rechnet

Auch ein guter Prompt garantiert keinen korrekten Code, daher folgt auf das Generieren das Verstehen. Dabei hilft eine feste Lesereihenfolge, die von außen nach innen vorgeht: zuerst die Signatur mit Parametern und Rückgabetyp, dann der Docstring, danach vorhandene Tests und erst

zuletzt die Implementierung. Signatur und Docstring beschreiben, was eine Funktion verspricht, während die Implementierung beschreibt, wie sie es umsetzt. Stimmt bereits das Versprechen nicht, ist die Umsetzung zweitrangig.

Für unser Beispiel lassen sich daraus drei Prüffragen ableiten. Die erste lautet, ob die Parameternamen die Einheiten tragen, die zweite, ob der Docstring die Einheit des Ergebnisses nennt. Bei `biegespannung_v1(b, h, M)` scheitert schon die erste Frage, denn die Namen sagen nichts über Einheiten aus, und ein Docstring fehlt ganz. Bei `biegespannung(b_mm, h_mm, M_knm)` stehen die Einheiten in den Namen, und der Docstring nennt N/mm² als Ergebnis. Die dritte Frage übernimmt die Rolle eines Tests, indem sie das Versprechen an einem Wert prüft, den wir unabhängig von der KI kennen: Mit b = 200 mm, h = 400 mm und M = 150 kNm muss die Funktion 28,125 N/mm² liefern. Erst wenn alle drei Fragen mit Ja beantwortet sind, lohnt sich der genaue Blick in die Implementierung; liefert die Funktion etwas anderes, ergänzen wir den Prompt um den fehlenden Constraint. Wie sich solche Prüfwerte als automatische Unit Tests formulieren lassen, behandeln wir in `08a_UnitTest`.

KI schreibt den Code, Sie verantworten ihn

Aus dem Zyklus ergibt sich ein neues Rollenverständnis. Die KI übernimmt die Rolle einer Junior-Entwicklerin, die schnell Code liefert, und wir übernehmen die Rolle der Senior-Entwicklerin oder des Senior-Entwicklers, die diesen Code abnehmen. Eine gute Senior-Entwicklerin liest den Code gründlich, stellt Fragen, fordert Änderungen und gibt erst dann ihre Freigabe.

Konkret umfasst diese Abnahme drei Rollen. Als Auftraggeber formulieren wir die Anforderung so präzise, dass die KI passenden Code erzeugen kann; im Beispiel fehlten genau hier die Einheiten. Als Leser verstehen wir den generierten Code mit seinen Annahmen und Grenzen, im Beispiel also die Division von kNm durch mm³ im Datenfluss. Als Verifizierer prüfen wir, ob das Ergebnis die Anforderung erfüllt, im Beispiel durch die Handrechnung mit 28,125 N/mm². Die Verantwortung für den Code bleibt damit beim Menschen, denn KI-generierter Code entbindet nicht von der Haftung für das Ergebnis.

Dann prüft eben der Mensch? → Die Kapazitätsfalle

Wenn die KI nicht allein arbeiten kann, liegt es nahe, die Prüfung einfach dem Menschen zu überlassen. Doch auch das stößt an Grenzen, weshalb wir hier von einer Kapazitätsfalle sprechen. Das Prüfen von KI-Output ist ermüdend, was unter dem Begriff *AI Fatigue* diskutiert wird, und es ist alles andere als trivial. In der Softwareentwicklung wächst die Menge an KI-generiertem Code schneller als die Kapazität für Reviews; auf GitHub hat sich die Zahl der Pull Requests vervierfacht [9].

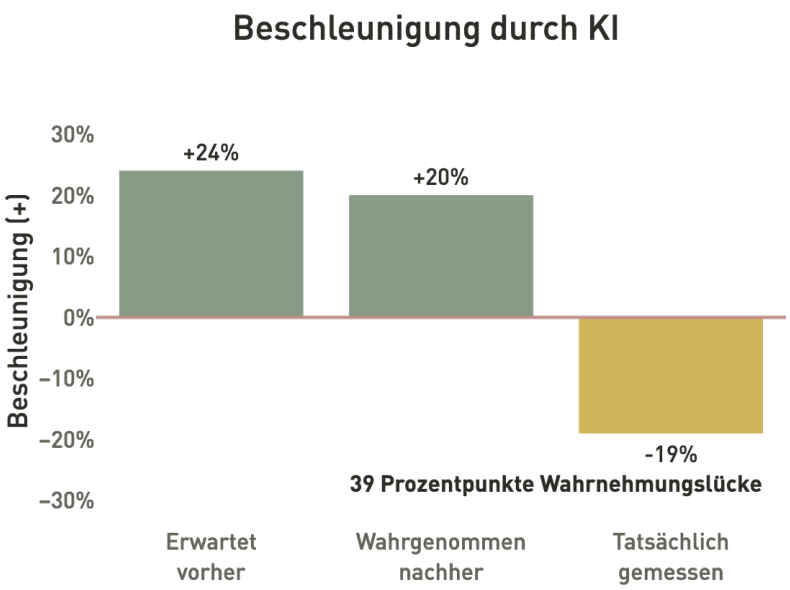


Abbildung 3: Erwartete, wahrgenommene und gemessene Beschleunigung durch KI in der METR-Studie [10]

Hinzu kommt, dass wir den eigenen Nutzen von KI schlecht einschätzen. Die METR-Studie ließ 16 erfahrene Open-Source-Entwicklerinnen und -Entwickler an 246 echten Issues arbeiten, wobei für jedes Issue zufällig festgelegt wurde, ob KI-Werkzeuge erlaubt waren [10]. Das Diagramm zeigt die drei zentralen Kennwerte der Studie. Vorab erwarteten die Teilnehmenden eine Beschleunigung um 24 Prozent, und auch nach der Arbeit schätzten sie die erlebte Beschleunigung noch auf 20 Prozent. Gemessen wurde jedoch eine Verlangsamung um 19 Prozent, sodass zwischen gefühlter und gemessener Produktivität 39 Prozentpunkte liegen. Wer sich auf das eigene Gefühl verlässt, prüft also womöglich weniger gründlich, als die Situation erfordert.

Und wer kann morgen noch prüfen? → Die Fähigkeitsfalle

Die dritte Falle betrifft die Zukunft, denn selbst wenn wir heute gründlich prüfen, müssen wir diese Fähigkeit auch morgen noch besitzen. Nutzende überschätzen ihre Fähigkeit, KI-Output zu beurteilen [11]. Zugleich führt häufige KI-Nutzung zu *Cognitive Offloading*, also zum Auslagern von Denkarbeit an das Werkzeug, und geht mit sinkendem kritischem Denken einher [12].

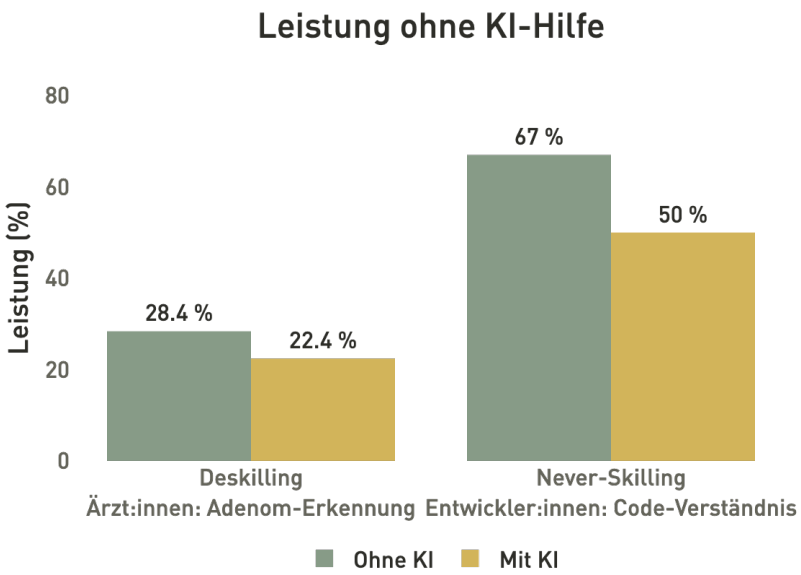


Abbildung 4: Leistung ohne KI-Hilfe: Deskilling bei Ärztinnen und Ärzten [13] und Never-Skilling beim Code-Verständnis [14]

Das Diagramm zeigt zwei Folgen davon, jeweils gemessen an der Leistung ohne KI-Hilfe. Beim *Deskilling* verlieren Erfahrene Fähigkeiten, die sie bereits beherrschten: In einer Studie zur Koloskopie sank die Adenom-Erkennungsrate von Ärztinnen und Ärzten bei Untersuchungen ohne KI von 28,4 auf 22,4 Prozent, nachdem sie zuvor mit KI-Unterstützung gearbeitet hatten [13]. Beim *Never-Skilling* erwerben Einsteigerinnen und Einsteiger die Grundlagen gar nicht erst [15]. Im Bereich Programmierung erreichten Entwicklerinnen und Entwickler, die sich mit KI-Hilfe in neue Inhalte eingearbeitet hatten, im anschließenden Test zum Code-Verständnis 50 statt 67 Prozent [14].

Zusammen ergibt sich ein Teufelskreis: Mehr KI-Assistenz führt zu Cognitive Offloading, dieses zu Deskilling und schließlich zu Never-Skilling. Für Studierende im ersten Semester ist das die eigentliche Begründung für den Aufbau dieses Kurses, denn wer KI-generierten Code prüfen will, muss zuerst selbst lernen, Code zu verstehen.

Zusammenfassung

Warum ist all das für Ingenieurinnen und Ingenieure wichtig? Eine Antwort gibt die Wissenstreppe nach North [16], die die Stufen von verfügbaren Daten bis zu besseren Ergebnissen ordnet. Aus Zeichen werden durch Syntax Daten, aus Daten durch Semantik Information und aus Information durch Vernetzung mit Kontext, Erfahrung und Erwartung Wissen. Diese unteren Stufen bilden explizites, theoretisches Wissen. Erst mit Anwendungsbezug wird aus Wissen Können, mit Motivation Handeln, durch richtiges Handeln Kompetenz und durch Einzigartigkeit schließlich Wettbewerbsfähigkeit; diese oberen Stufen bilden implizites, operatives Wissen.

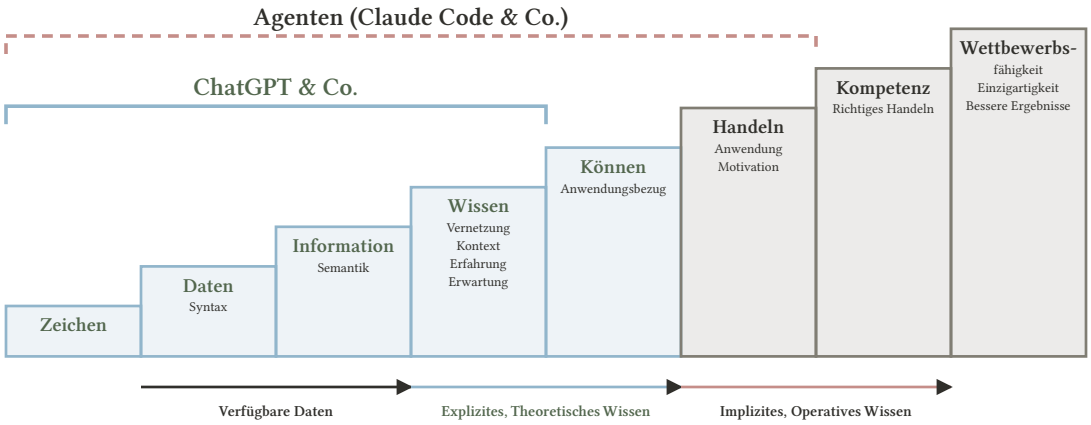


Abbildung 5: Wissenstreppe nach North mit den Stufen, die Sprachmodelle und Agenten abdecken

ChatGPT und vergleichbare Modelle decken die Stufen von den Zeichen bis zum Wissen ab. Agenten wie Claude Code übernehmen zusätzlich das Handeln, denn sie führen Code aus, ändern Dateien und starten Tests. Ob eine Handlung richtig ist, beurteilen sie jedoch nicht verlässlich, und genau an dieser Stelle beginnt die Kompetenz. Für unser Beispiel heißt das: Den Code für die Biegespannung schreibt die KI in Sekunden, aber zu erkennen, dass $2{,}8{\cdot}10^{-5}$ N/mm² nicht stimmen kann, und die Anforderung so zu formulieren, dass 28,125 N/mm² herauskommt, bleibt unsere Aufgabe. Diese Kompetenz ist zugleich unser entscheidender Vorteil gegenüber dem Werkzeug.

Die wichtigsten Punkte dieses Kapitels lassen sich so zusammenfassen. Sprachmodelle erzeugen plausiblen, aber nicht garantiert korrekten Code, weil ihr Training Wortwahrscheinlichkeiten belohnt und keine Rechnung prüft. Ein gut strukturierter Prompt mit Rolle, Kontext, Aufgabe, Constraint und Ausgabeformat ist eine Anforderungsspezifikation. Im agentischen Zyklus übernimmt die KI das Generieren, während wir als Auftraggeber, Leser und Verifizierer für Anforderung und Ergebnis verantwortlich bleiben.

Quiz

Referenzen

Bibliographie

[1] A. Karpathy, „There's a new kind of coding I call ``vibe coding''“. Februar 2025.

[2] A. Vaswani *u. a.*, „Attention is All You Need“, *NeurIPS*, Bd. 30, 2017, [Online]. Verfügbar unter: https://arxiv.org/abs/1706.03762

[3] J. Ploennigs *u. a.*, „Building Foundation Models - Potentials, Challenges and Research Directions for Using LLM and LVM in AEC“, in *EC3*, 2025.

[4] L. Ouyang *u. a.*, „Training language models to follow instructions with human feedback“, *arXiv preprint arXiv:2203.02155*, 2022, [Online]. Verfügbar unter: https://arxiv.org/abs/2203.02155

[5] V. Pursnani, Y. Sermet, M. Kurt, und I. Demir, „Performance of ChatGPT on the US fundamentals of engineering exam“, *Computers and Education: AI*, Bd. 5, S. 100183, 2023, doi: https://doi.org/10.1016/j.caeai.2023.100183.

[6] L. Oblitas, S. Adeeb, und C. Cruz-Noguez, „Performance of AI Chatbots on the Fundamentals of Engineering Civil Exam“, *Discover Artificial Intelligence*, Bd. 5, S. 378, 2025, doi: 10.1007/s44163-025-00646-6.

[7] T. B. Brown *u. a.*, „Language Models are Few-Shot Learners“, *NeurIPS*, Bd. 33, S. 1877–1901, 2020, [Online]. Verfügbar unter: https://arxiv.org/abs/2005.14165

[8] M. Chen, J. Tworek, H. Jun, Q. Yuan, und others, „Evaluating Large Language Models Trained on Code“, *arXiv preprint arXiv:2107.03374*, 2021, [Online]. Verfügbar unter: https://arxiv.org/abs/2107.03374

[9] CodeRabbit, „GitHub Gives Maintainers a Throttle for the AI Pull Request“. Zugegriffen: 17. September 2026. [Online]. Verfügbar unter: https://www.coderabbit.ai/blog/github-gives-maintainers-a-throttle-for-the-ai-pull-request

[10] METR, „Measuring the Impact of Early-2025 AI on Experienced Open-Source Developer Productivity“. [Online]. Verfügbar unter: https://metr.org/blog/2025-07-10-early-2025-ai-experienced-os-dev-study/

[11] D. Fernandes, T. Kosch, und P. W. Woźniak, „AI makes you smarter but none the wiser: The disconnect between performance and metacognition“, *Computers in Human Behavior*, 2026, doi: 10.1016/j.chb.2025.108779.

[12] M. Gerlich, „AI Tools in Society: Impacts on Cognitive Offloading and the Future of Critical Thinking“, *Societies*, Bd. 15, Nr. 1, S. 6, 2025, doi: 10.3390/soc15010006.

[13] K. Budzyń, M. Romańczyk, D. Kitala, und others, „Endoscopist deskilling risk after exposure to artificial intelligence in colonoscopy: a multicentre, observational study“, *The Lancet Gastroenterology & Hepatology*, Bd. 10, Nr. 10, S. 896–903, 2025, doi: 10.1016/S2468-1253(25)00133-5.

[14] J. H. Shen und A. Tamkin, „How AI Impacts Skill Formation“, *arXiv:2601.20245*, 2026.

[15] Y. Ke, L. Jin, J. C. L. Ong, und others, „AI-induced never-skilling in medical education“, *Nature Medicine*, Bd. 32, Nr. 6, S. 1997–2006, 2026, doi: 10.1038/s41591-026-04438-y.

[16] K. North, *Wissensorientierte Unternehmensführung: Wertschöpfung durch Wissen*, 5. Aufl. Wiesbaden: Gabler, 2011.